StyleSense: Business Analytics

Answering 8 Business Questions from the Data

StyleSense is a growing clothing retailer selling through physical stores and online. Leadership needs clear answers to act on. This notebook works through 8 business questions drawn directly from the problems identified in the cleaning notebook.

Each question is labeled with the type of analysis it performs:

Question	Business Question	Analysis Type
Q1	Which products are the most profitable?	Descriptive
Q2	Which customer segment spends the most?	Descriptive
Q3	Do discounts actually increase sales?	Diagnostic
Q4	How does each sales channel perform?	Descriptive + Diagnostic
Q5	Is marketing spend generating returns?	Diagnostic + Prescriptive
Q6	When should we restock seasonal products?	Predictive
Q7	Which products are frequently out of stock?	Diagnostic + Prescriptive
Q8	What is our true profit after all costs?	Descriptive + Prescriptive

The four analysis types used in this notebook:

- **Descriptive:** What happened? Summarises what the data shows.
- **Diagnostic:** Why did it happen? Looks for causes and patterns behind the numbers.
- **Predictive:** What is likely to happen next? Uses historical patterns to forecast.
- **Prescriptive:** What should we do? Turns findings into clear recommendations.

In [1]:
```
# Library imports
import pandas as pd                    # data loading, grouping, and table operations
import numpy as np                     # numeric calculations
import matplotlib.pyplot as plt        # charts and visualisations
import matplotlib.ticker as mticker    # number formatting on chart axes
```

```python
import matplotlib.patches as mpatches  # custom legend elements
import seaborn as sns                  # heatmaps and styled charts

# Chart styling
sns.set_theme(style="whitegrid")
plt.rcParams['figure.dpi'] = 110

# Colour palette used consistently across all charts
BLUE   = "#1f77b4"
GREEN  = "#2ca02c"
ORANGE = "#ff7f0e"
RED    = "#d62728"
PURPLE = "#9467bd"
TEAL   = "#17becf"
```

In [2]:
```python
# Load the feature-engineered datasets produced in the Feature Engineering notebook.
# These contain all the original cleaned columns plus new columns like
# margin_erosion, RFM_segment, is_near_stockout, days_until_stockout, and is_fixed_cost.
sales     = pd.read_csv('sales_featured.csv',     parse_dates=['date'])
customers = pd.read_csv('customers_featured.csv', parse_dates=['first_purchase_date', 'last_purchase_date'])
products  = pd.read_csv('products_featured.csv')
expenses  = pd.read_csv('expenses_featured.csv',  parse_dates=['date'])
inventory = pd.read_csv('inventory_featured.csv', parse_dates=['date'])
```

In [3]:
```python
# Add a 'month' period column to sales, expenses, and inventory.
# dt.to_period('M') converts a date like 2024-03-15 into 2024-03 so we can group by month.
sales['month']     = sales['date'].dt.to_period('M')
expenses['month']  = expenses['date'].dt.to_period('M')
inventory['month'] = inventory['date'].dt.to_period('M')
```

In [4]:
```python
print("Datasets loaded:")
print(f"  sales     : {sales.shape}")
print(f"  customers : {customers.shape}")
print(f"  products  : {products.shape}")
print(f"  expenses  : {expenses.shape}")
print(f"  inventory : {inventory.shape}")
```

```
Datasets loaded:
  sales     : (9989, 37)
  customers : (200, 24)
  products  : (25, 12)
  expenses  : (1322, 10)
  inventory : (13675, 17)
```

Which Products Are the Most Profitable?

Analysis type: Descriptive (What happened?)

Not all products that sell well are actually profitable. This section measures each product by two things: how much total net profit it generated over 18 months, and how much of its theoretical profit was lost to discounts.

The difference between the profit margin on the price list and the margin actually achieved after discounts is called margin erosion. A product with high margin erosion is being discounted so heavily that the business is earning far less than it should be.

In [5]:
```
# Group sales by product and calculate total profit, units, revenue, and margin metrics.
# catalogue_margin_pct and margin_erosion were added in the Feature Engineering notebook.
product_profit = sales.groupby('product_name').agg(
    total_net_profit     = ('net_profit',            'sum'),
    total_units_sold     = ('quantity_sold',          'sum'),
    total_revenue        = ('total_sale',             'sum'),
    transactions         = ('sale_id',                'count'),
    avg_realised_margin  = ('profit_margin_pct',      'mean'),
    avg_catalogue_margin = ('catalogue_margin_pct',   'mean'),
    avg_margin_erosion   = ('margin_erosion',         'mean')
).reset_index()

# Profit per unit: how much net profit does each unit sold generate on average
product_profit['avg_profit_per_unit'] = (product_profit['total_net_profit'] / product_profit['total_units_sold']).rou
product_profit['avg_realised_margin'] = product_profit['avg_realised_margin'].round(1)
product_profit['avg_margin_erosion']  = product_profit['avg_margin_erosion'].round(1)

# Sort by total net profit so the most profitable product appears first
product_profit = product_profit.sort_values('total_net_profit', ascending=False)

print("Top 10 Most Profitable Products:")
```

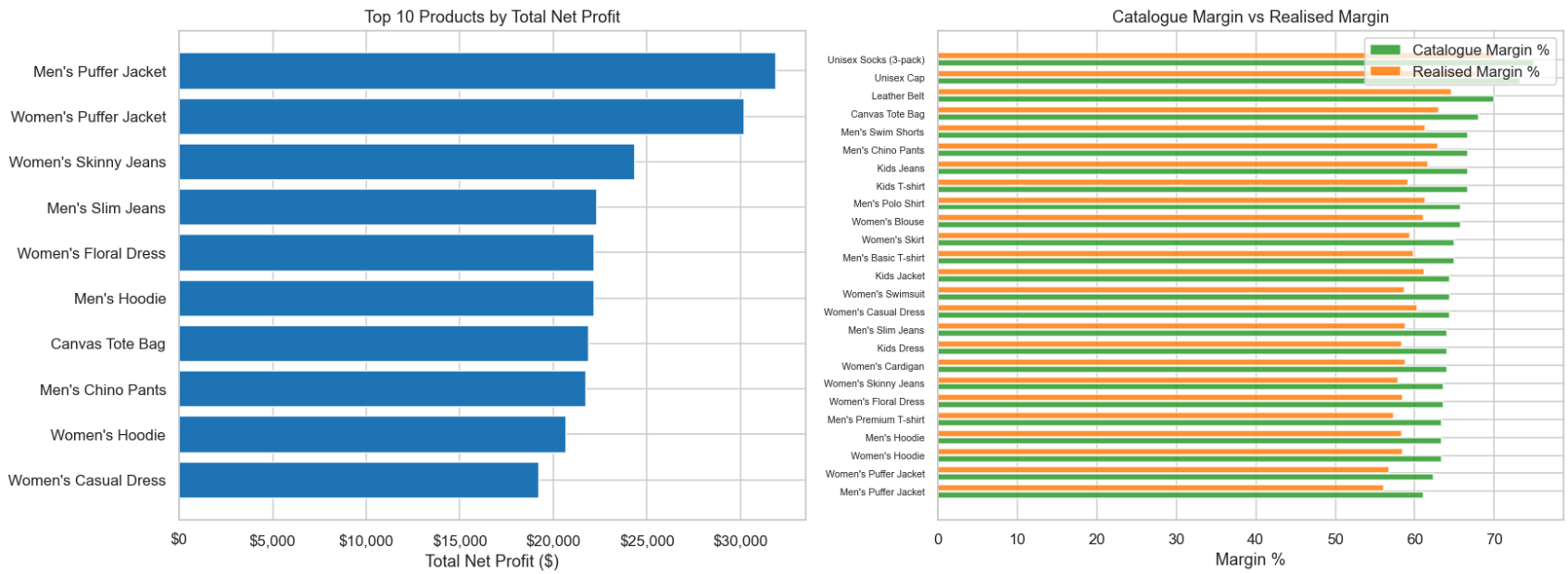
```
print(product_profit.head(10)[['product_name', 'total_net_profit', 'total_units_sold',
                                'avg_profit_per_unit', 'avg_realised_margin', 'avg_margin_erosion']].to_string(index=
```

```
Top 10 Most Profitable Products:
         product_name  total_net_profit  total_units_sold  avg_profit_per_unit  avg_realised_margin  avg_margin_erosi
on
  Men's Puffer Jacket          31885.20               661                48.24                 56.1
5.0
Women's Puffer Jacket          30211.28               662                45.64                 56.8
5.5
 Women's Skinny Jeans          24371.70               804                30.31                 57.9
5.7
     Men's Slim Jeans          22316.97               803                27.79                 58.8
5.2
 Women's Floral Dress          22182.88               726                30.55                 58.5
5.1
         Men's Hoodie          22168.60               673                32.94                 58.3
5.0
      Canvas Tote Bag          21880.35              1481                14.77                 63.0
5.0
    Men's Chino Pants          21748.90               816                26.65                 62.9
3.8
       Women's Hoodie          20681.06               627                32.98                 58.5
4.8
 Women's Casual Dress          19238.85               744                25.86                 60.3
4.1
```

In [6]:

```
# Chart 1 (left): horizontal bar chart of the top 10 products by total net profit.
# Chart 2 (right): grouped bars comparing catalogue margin vs realised margin per product.
# The gap between green and orange bars is the margin erosion for each product.

top10 = product_profit.head(10)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Left chart: top 10 products ranked by total net profit
axes[0].barh(top10['product_name'][::-1], top10['total_net_profit'][::-1], color=BLUE)
axes[0].set_xlabel("Total Net Profit ($)")
axes[0].set_title("Top 10 Products by Total Net Profit")
axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))

# Right chart: all products with catalogue margin (green) vs realised margin (orange)
```

```
prod_sorted = product_profit.sort_values('avg_catalogue_margin')
positions   = range(len(prod_sorted))

axes[1].barh([i - 0.2 for i in positions], prod_sorted['avg_catalogue_margin'], 0.35,
             label='Catalogue Margin %', color=GREEN, alpha=0.85)
axes[1].barh([i + 0.2 for i in positions], prod_sorted['avg_realised_margin'], 0.35,
             label='Realised Margin %', color=ORANGE, alpha=0.85)
axes[1].set_yticks(list(positions))
axes[1].set_yticklabels(prod_sorted['product_name'], fontsize=7)
axes[1].set_title("Catalogue Margin vs Realised Margin")
axes[1].set_xlabel("Margin %")
axes[1].legend()

plt.tight_layout()
plt.show()
```



In [7]:

```
# Profit breakdown by product category.
# This shows whether Men, Women, Kids, or All-season products generate the most profit overall.
category_profit = sales.groupby('category').agg(
    total_net_profit = ('net_profit',  'sum'),
    total_revenue    = ('total_sale',  'sum')
).reset_index()
```

```python
# Calculate the profit margin at the category level
category_profit['profit_margin_pct'] = (category_profit['total_net_profit'] / category_profit['total_revenue'] * 100)
category_profit = category_profit.sort_values('total_net_profit', ascending=False)

print("Profitability by Category:")
print(category_profit.to_string(index=False))
```

```
Profitability by Category:
category  total_net_profit  total_revenue  profit_margin_pct
   Women         179111.50      304622.14               58.8
     Men         140967.98      238494.33               59.1
     All          59522.76       91135.73               65.3
    Kids          15358.15       25160.44               61.0
   women           3462.81        5818.77               59.5
     men           2741.90        4375.89               62.7
     all           1381.08        2042.07               67.6
    kids            293.57         459.57               63.9
```

Finding

The two **Puffer Jackets** are the most profitable products.

- **Men's Puffer Jacket:** ~ $32,000 net profit (18 months)
- **Women's Puffer Jacket:** ~ $30,000 net profit

They outperform the next group by about $5,000$–**7,000**.

The next tier includes:

- Women's Skinny Jeans
- Men's Slim Jeans
- Women's Floral Dress

Each generates roughly $22,000$–**25,000** in net profit.

Key Insight

The gap between **catalogue margin (green)** and **realised margin (orange)** is small and consistent across all **25 products**, showing minimal margin erosion.

Although **Puffer Jackets** have lower margins (**58–60%**), they generate the **highest profit due to high sales volume**.

Accessories like **Unisex Socks, Unisex Cap, and Leather Belt Carry** have **higher margins (65–70%)** but lower total profit because they sell in **smaller volumes**.

Conclusion

Sales volume is the main driver of total profitability, not margin percentage.

Which Customer Segment Spends the Most?

Analysis type: Descriptive (What happened?)

StyleSense has 200 customers. This section compares how much different customer groups spend, using three different ways of grouping them:

1. **CRM segment** (how the customer was registered): Loyal, New, Seasonal, Occasional
2. **Engagement segment** (based on how many orders they have placed): Low, Mid, High Engagement
3. **RFM segment** (based on how recently, how often, and how much they buy): Champion, Loyal, Potential, At Risk, Lost

All three views look at the same 200 customers but from different angles. Together they show who is most valuable and, crucially, who is at risk of leaving.

In [8]:
```
# Average and total spend grouped by the CRM registration segment.
# This is the segment label that came with the raw customer data.
seg_spend = customers.groupby('segment').agg(
    customer_count = ('customer_id', 'count'),
    avg_spent      = ('total_spent',  'mean'),
    total_spent    = ('total_spent',  'sum'),
    avg_orders     = ('num_orders',   'mean')
).reset_index()

seg_spend['avg_spent']  = seg_spend['avg_spent'].round(2)
seg_spend['avg_orders'] = seg_spend['avg_orders'].round(1)
seg_spend = seg_spend.sort_values('avg_spent', ascending=False)
```

```
print("Spending by CRM Segment:")
print(seg_spend.to_string(index=False))
```

```
Spending by CRM Segment:
       segment  customer_count  avg_spent  total_spent  avg_orders
         Loyal              36    4669.46    168100.54        51.0
Bargain Hunter              18    4647.88     83661.76        49.0
    Occasional              55    2988.80    164384.17        50.7
      Seasonal              35    2826.54     98929.03        50.9
           New              56    2804.13    157031.26        48.2
```

In [9]:

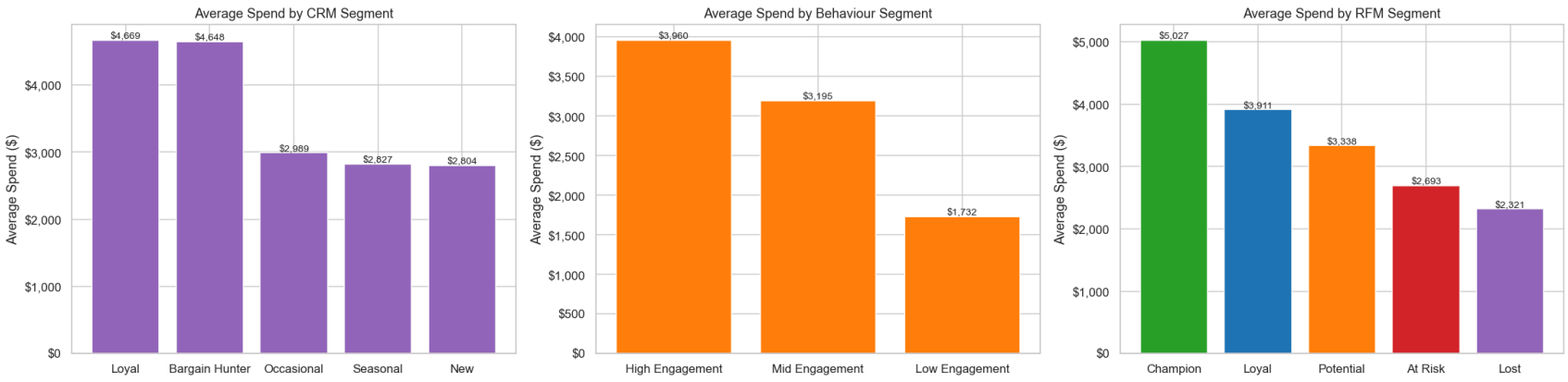
```
# Average and total spend grouped by the behaviour-based derived_segment.
# This segment was built during data cleaning based on how many orders each customer placed.
# Low Engagement: fewer than 35 orders, Mid: 35 to 54, High: 55 or more.
derived_spend = customers.groupby('derived_segment').agg(
    customer_count = ('customer_id', 'count'),
    avg_spent      = ('total_spent',  'mean'),
    total_spent    = ('total_spent',  'sum'),
    avg_orders     = ('num_orders',   'mean')
).reset_index()

derived_spend['avg_spent']  = derived_spend['avg_spent'].round(2)
derived_spend['avg_orders'] = derived_spend['avg_orders'].round(1)
derived_spend = derived_spend.sort_values('avg_spent', ascending=False)

print("Spending by Behaviour-Based Segment:")
print(derived_spend.to_string(index=False))
```

```
Spending by Behaviour-Based Segment:
derived_segment  customer_count  avg_spent  total_spent  avg_orders
High Engagement              47    3960.22    186130.51        60.0
 Mid Engagement             151    3195.45    482512.22        47.1
 Low Engagement               2    1732.02      3464.03        31.0
```

In [10]:

```
# Three bar charts side by side: CRM segment, behaviour segment, and RFM segment.
# Each shows average spend so the bars are comparable regardless of group size.
fig, axes = plt.subplots(1, 3, figsize=(20, 5))

# Left chart: average spend by CRM registration segment
axes[0].bar(seg_spend['segment'], seg_spend['avg_spent'], color=PURPLE)
axes[0].set_title("Average Spend by CRM Segment")
axes[0].set_ylabel("Average Spend ($)")
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
```

```python
for bar, val in zip(axes[0].patches, seg_spend['avg_spent']):
    axes[0].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 20,
                 f"${val:,.0f}", ha='center', fontsize=9)

# Centre chart: average spend by behaviour-based segment
axes[1].bar(derived_spend['derived_segment'], derived_spend['avg_spent'], color=ORANGE)
axes[1].set_title("Average Spend by Behaviour Segment")
axes[1].set_ylabel("Average Spend ($)")
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
for bar, val in zip(axes[1].patches, derived_spend['avg_spent']):
    axes[1].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 20,
                 f"${val:,.0f}", ha='center', fontsize=9)

# Right chart: average spend by RFM segment (built in Feature Engineering)
rfm_spend = customers.groupby('RFM_segment').agg(
    customer_count = ('customer_id', 'count'),
    avg_spent      = ('total_spent',  'mean')
).reset_index()

# Order the RFM segments from highest to lowest value
seg_order = ['Champion', 'Loyal', 'Potential', 'At Risk', 'Lost']
rfm_spend['RFM_segment'] = pd.Categorical(rfm_spend['RFM_segment'], categories=seg_order, ordered=True)
rfm_spend = rfm_spend.sort_values('RFM_segment')

colors_rfm = [GREEN, BLUE, ORANGE, RED, PURPLE]
axes[2].bar(rfm_spend['RFM_segment'], rfm_spend['avg_spent'], color=colors_rfm)
axes[2].set_title("Average Spend by RFM Segment")
axes[2].set_ylabel("Average Spend ($)")
axes[2].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
for bar, val in zip(axes[2].patches, rfm_spend['avg_spent']):
    axes[2].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 20,
                 f"${val:,.0f}", ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```



In [11]:
```
# Which product categories does each CRM segment prefer?
# This tells marketing which products to feature in campaigns for each group.
seg_fav = customers.groupby(['segment', 'favourite_category']).size().reset_index(name='count')
seg_fav = seg_fav.sort_values(['segment', 'count'], ascending=[True, False])
top_fav = seg_fav.groupby('segment').head(3)

print("Top 3 Favourite Categories per CRM Segment:")
print(top_fav.to_string(index=False))
```

```
Top 3 Favourite Categories per CRM Segment:
       segment favourite_category  count
Bargain Hunter              Women     10
Bargain Hunter                Men      8
         Loyal                Men     21
         Loyal              Women     15
           New              Women     37
           New                Men     18
           New                All      1
    Occasional              Women     27
    Occasional                Men     25
    Occasional                All      3
      Seasonal                Men     19
      Seasonal              Women     16
```

In [12]:
```
# Identify the top At Risk customers by total spend.
# These are customers who were high-value but have not purchased recently.
# This list is the starting point for a targeted win-back campaign.
at_risk_customers = customers[customers['RFM_segment'] == 'At Risk'].copy()
at_risk_customers = at_risk_customers.sort_values('total_spent', ascending=False)
```

```python
print("Top 10 At Risk Customers by Total Spend (win-back priority list):")
print(at_risk_customers[['customer_id', 'total_spent', 'num_orders',
                         'recency_days', 'favourite_category']].head(10).to_string(index=False))
```

```
Top 10 At Risk Customers by Total Spend (win-back priority list):
customer_id  total_spent  num_orders  recency_days favourite_category
       C053      4396.63          43            26              Women
       C174      3912.64          47            30                Men
       C063      3528.49          39            30                Men
       C192      3474.68          47            30              Women
       C166      3410.99          46            28              Women
       C041      3232.88          47            20              Women
       C015      3209.94          40            12              Women
       C097      3107.70          46            13              Women
       C104      3079.52          43            15              Women
       C060      3030.23          48            26              Women
```

Findings:

CRM Segment Spending

Two segments lead total spending:

- **Loyal Customers:** ~$4,669 average spend
- **Bargain Hunters:** ~$4,648 average spend

They are nearly tied as the **highest-spending groups**.

Other segments spend less:

- **Occasional:** ~$2,989
- **Seasonal:** ~$2,827
- **New:** ~$2,804

These groups cluster around $2,800$–**3,000**.

Engagement Impact

Spending strongly increases with engagement:

- **High Engagement:** ~$3,960
- **Mid Engagement:** ~$3,195
- **Low Engagement:** ~$1,732

This shows **engagement level is a key driver of customer spending**.

RFM Segments

- **Champions:** ~$5,027 (highest overall)
- **Loyal:** ~$3,911
- **At Risk:** ~$2,693
- **Lost:** ~$2,321

The **win-back list** highlights high-value customers in the **At Risk** group to prioritize for re-engagement.

Do Discounts Actually Increase Sales?

Analysis type: Diagnostic (Why did it happen?)

StyleSense applies discounts of 0%, 5%, 10%, 15%, and 20% across its product range. The question is whether offering a bigger discount actually brings in more transactions and more profit, or whether it simply reduces the price without meaningfully lifting sales.

This section compares each discount tier across four measures: how many transactions occurred, how much revenue each transaction generated on average, what the profit margin was, and how often items were returned. A discount is only worth offering if the extra volume it generates more than compensates for the margin it removes.

In [13]:
```
# Group transactions by discount tier.
# For each tier we want to know: how many transactions, how much revenue,
```

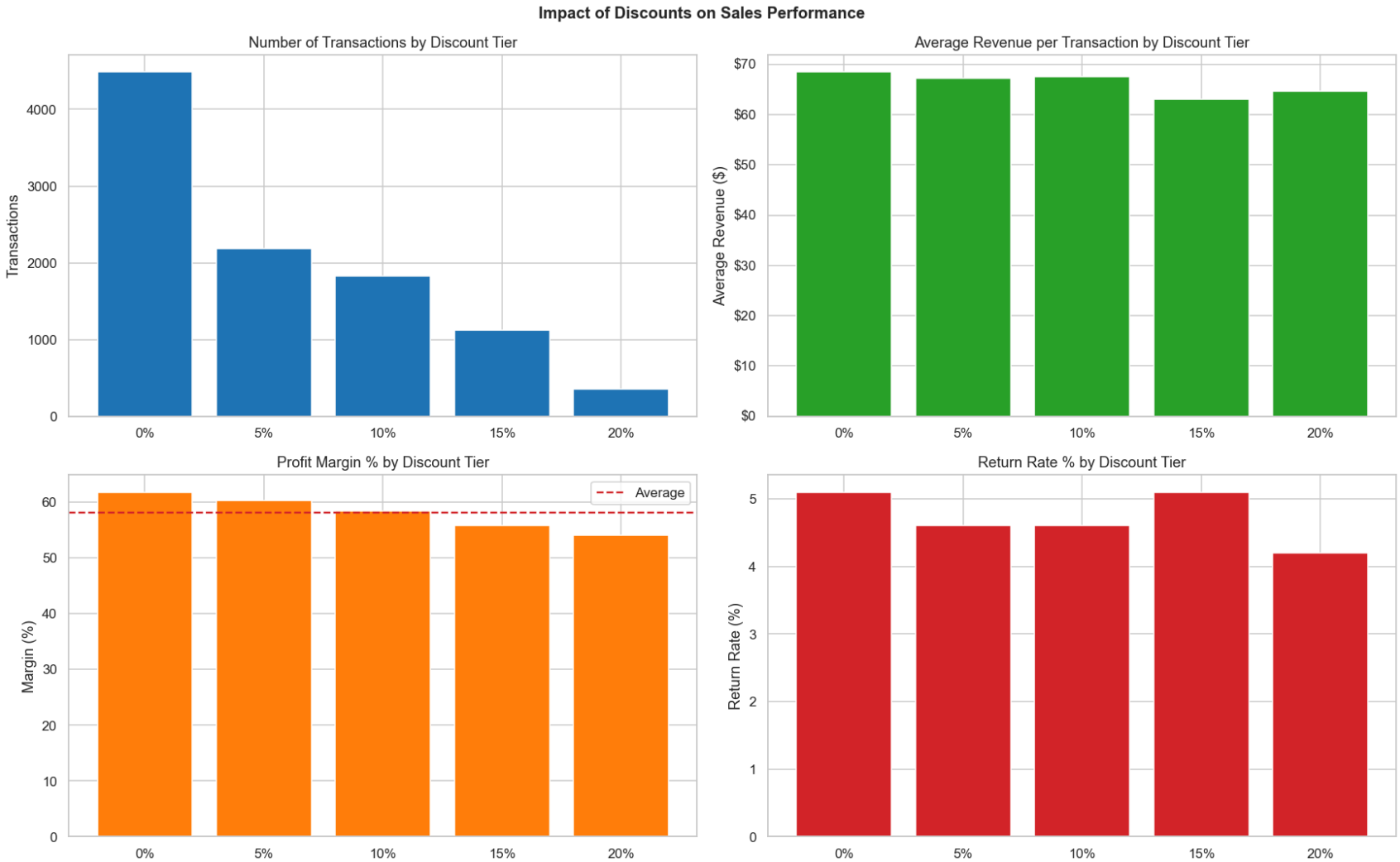
```python
# what was the profit margin, how often were items returned, and how much margin was eroded.
discount_analysis = sales.groupby('discount_pct').agg(
    transactions       = ('sale_id',          'count'),
    avg_units_sold     = ('quantity_sold',     'mean'),
    total_units_sold   = ('quantity_sold',     'sum'),
    total_revenue      = ('total_sale',        'sum'),
    total_net_profit   = ('net_profit',        'sum'),
    return_rate        = ('is_returned',       'mean'),
    avg_margin_erosion = ('margin_erosion',    'mean')
).reset_index()

# Calculate derived metrics per tier
discount_analysis['avg_revenue_per_tx'] = (discount_analysis['total_revenue']    / discount_analysis['transactions'])
discount_analysis['avg_profit_per_tx']  = (discount_analysis['total_net_profit'] / discount_analysis['transactions'])
discount_analysis['profit_margin_pct']  = (discount_analysis['total_net_profit'] / discount_analysis['total_revenue'])
discount_analysis['return_rate']        = (discount_analysis['return_rate'] * 100).round(1)
discount_analysis['avg_margin_erosion'] = discount_analysis['avg_margin_erosion'].round(1)

print("Discount Tier Summary:")
print(discount_analysis[['discount_pct', 'transactions', 'avg_units_sold', 'avg_revenue_per_tx',
                         'avg_profit_per_tx', 'profit_margin_pct', 'return_rate',
                         'avg_margin_erosion']].to_string(index=False))
```

```
Discount Tier Summary:
 discount_pct  transactions  avg_units_sold  avg_revenue_per_tx  avg_profit_per_tx  profit_margin_pct  return_rate  a
vg_margin_erosion
            0          4489        1.811985               68.51              42.31               61.8          5.1
3.4
            5          2189        1.870260               67.15              40.48               60.3          4.6
4.7
           10          1829        1.926189               67.55              39.47               58.4          4.6
6.6
           15          1127        1.924579               63.08              35.22               55.8          5.1
9.0
           20           355        2.123944               64.64              34.96               54.1          4.2
10.9
```

In [14]:

```python
# Four charts showing the effect of each discount tier on key performance metrics.
fig, axes = plt.subplots(2, 2, figsize=(16, 10))

# Create a label like "0%", "5%", etc. for the x-axis
x = discount_analysis['discount_pct'].astype(str) + '%'
```

```python
# Chart 1: how many transactions occurred at each discount level
axes[0, 0].bar(x, discount_analysis['transactions'], color=BLUE)
axes[0, 0].set_title("Number of Transactions by Discount Tier")
axes[0, 0].set_ylabel("Transactions")

# Chart 2: average revenue earned per transaction at each tier
axes[0, 1].bar(x, discount_analysis['avg_revenue_per_tx'], color=GREEN)
axes[0, 1].set_title("Average Revenue per Transaction by Discount Tier")
axes[0, 1].set_ylabel("Average Revenue ($)")
axes[0, 1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))

# Chart 3: profit margin percentage at each tier with an average reference line
axes[1, 0].bar(x, discount_analysis['profit_margin_pct'], color=ORANGE)
axes[1, 0].set_title("Profit Margin % by Discount Tier")
axes[1, 0].set_ylabel("Margin (%)")
axes[1, 0].axhline(discount_analysis['profit_margin_pct'].mean(), color=RED,
                   linestyle='--', label='Average')
axes[1, 0].legend()

# Chart 4: percentage of transactions that were returned at each tier
axes[1, 1].bar(x, discount_analysis['return_rate'], color=RED)
axes[1, 1].set_title("Return Rate % by Discount Tier")
axes[1, 1].set_ylabel("Return Rate (%)")

plt.suptitle("Impact of Discounts on Sales Performance", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



Finding: The transaction volume chart tells the clearest story: the no-discount tier (0%) accounts for approximately 4,450 transactions, more than twice the 5% tier (2,200) and thirteen times the 20% tier (350). Higher discount tiers are not attracting more buyers. The customer base strongly prefers full-price purchasing.

Average revenue per transaction is almost identical across all five tiers, ranging from roughly $63 to 68$. Giving a larger discount does not increase the size of each order; it simply reduces the margin earned on a similar basket.

Profit margin declines steadily as discounts increase: approximately 62% at 0%, falling to around 54% at 20%. The 20% tier sits clearly below the average of 59%, shown by the red dashed line. Return rates are effectively flat across all five tiers, ranging from 4.2% to 5.1% with no consistent pattern. There is no evidence that discounts increase return rates. The overall conclusion is that higher discounts attract fewer customers at lower margins and bring no benefit in order size or return reduction.

How Does Each Sales Channel Perform?

Analysis type: Descriptive + Diagnostic (What happened? + Why?)

StyleSense sells through two channels: physical stores and online. This section compares the two channels across revenue, profit, average order size, return rates, and the discount behaviour of each channel's customers.

Understanding which channel is more profitable, and why, is essential before deciding where to invest in marketing or operations. A channel with higher revenue but lower profit margin may be costing more to operate than it returns.

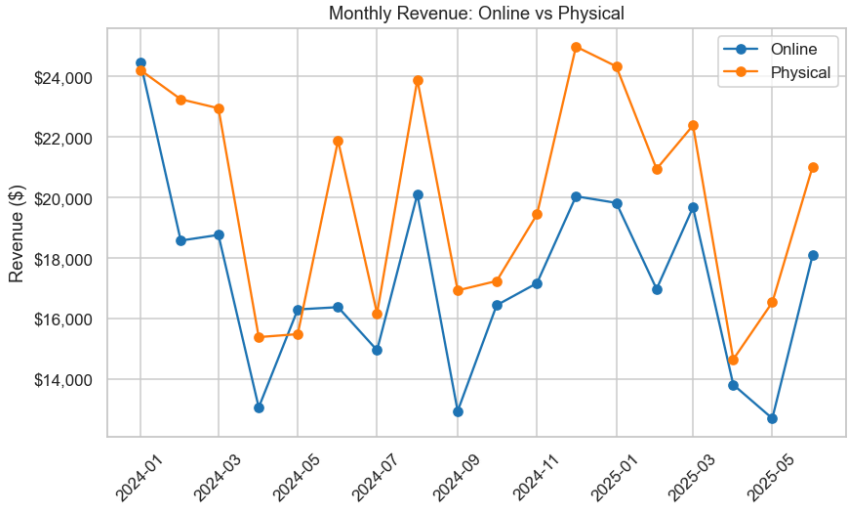
In [15]:
```
# Compare the two sales channels across all key performance metrics.
channel = sales.groupby('store_channel').agg(
    transactions       = ('sale_id',          'count'),
    total_revenue      = ('total_sale',        'sum'),
    total_net_profit   = ('net_profit',        'sum'),
    avg_order_value    = ('total_sale',        'mean'),
    avg_discount_pct   = ('discount_pct',      'mean'),
    return_rate        = ('is_returned',       'mean'),
    avg_units_sold     = ('quantity_sold',     'mean'),
    avg_margin_erosion = ('margin_erosion',    'mean')
).reset_index()

# Calculate profit margin at the channel level
channel['profit_margin_pct']  = (channel['total_net_profit'] / channel['total_revenue'] * 100).round(1)
channel['return_rate']        = (channel['return_rate'] * 100).round(1)
channel['avg_order_value']    = channel['avg_order_value'].round(2)
channel['avg_discount_pct']   = channel['avg_discount_pct'].round(1)
channel['avg_margin_erosion'] = channel['avg_margin_erosion'].round(1)

print("Channel Performance Summary:")
print(channel.to_string(index=False))
```

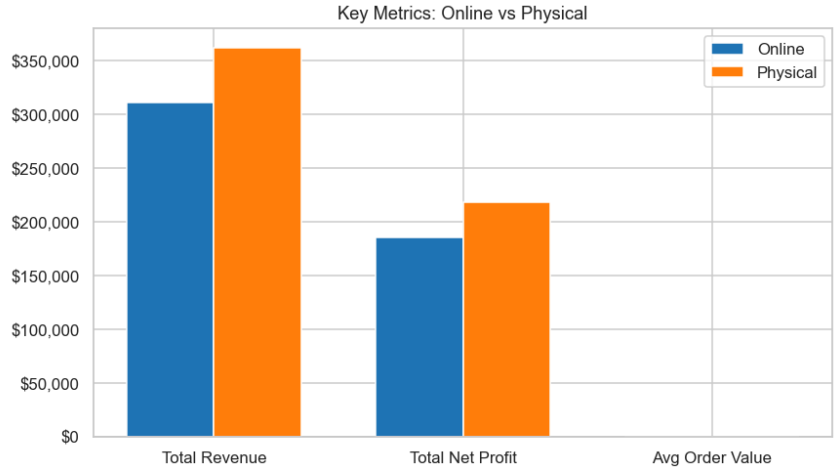
```
Channel Performance Summary:
store_channel  transactions  total_revenue  total_net_profit  avg_order_value  avg_discount_pct  return_rate  avg_uni
ts_sold  avg_margin_erosion  profit_margin_pct
       Online          4547      310378.98         185226.35            68.26               5.3          5.2
1.878821                 5.4               59.7
     Physical          5442      361729.96         217613.40            66.47               5.3          4.5
1.861632                 5.0               60.2
```

In [16]:

```python
# Left chart: monthly revenue trend for each channel over 18 months.
# Right chart: side-by-side bar comparison of revenue, profit, and average order value.
monthly_channel = sales.groupby(['month', 'store_channel'])['total_sale'].sum().reset_index()
monthly_channel['month_dt'] = monthly_channel['month'].dt.to_timestamp()

online   = monthly_channel[monthly_channel['store_channel'] == 'Online']
physical = monthly_channel[monthly_channel['store_channel'] == 'Physical']

fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Left chart: monthly revenue trend
axes[0].plot(online['month_dt'],   online['total_sale'],   label='Online',   color=BLUE,   marker='o')
axes[0].plot(physical['month_dt'], physical['total_sale'], label='Physical', color=ORANGE, marker='o')
axes[0].set_title("Monthly Revenue: Online vs Physical")
axes[0].set_ylabel("Revenue ($)")
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
axes[0].legend()
axes[0].tick_params(axis='x', rotation=45)

# Right chart: grouped bars for total revenue, total net profit, and average order value
metrics       = ['total_revenue', 'total_net_profit', 'avg_order_value']
labels        = ['Total Revenue', 'Total Net Profit', 'Avg Order Value']
positions     = range(len(metrics))
bar_width     = 0.35
online_vals   = [channel[channel['store_channel'] == 'Online'][m].values[0]   for m in metrics]
physical_vals = [channel[channel['store_channel'] == 'Physical'][m].values[0] for m in metrics]

axes[1].bar([i - bar_width / 2 for i in positions], online_vals,   bar_width, label='Online',   color=BLUE)
axes[1].bar([i + bar_width / 2 for i in positions], physical_vals, bar_width, label='Physical', color=ORANGE)
axes[1].set_xticks(list(positions))
axes[1].set_xticklabels(labels)
axes[1].set_title("Key Metrics: Online vs Physical")
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, _: f"${v:,.0f}"))
axes[1].legend()
```

```
plt.tight_layout()
plt.show()
```





In [17]:

```
# Payment method breakdown by channel.
# This shows whether online and physical customers pay differently,
# which can be useful when evaluating payment processing costs.
payment_channel = sales.groupby(['store_channel', 'payment_method']).size().reset_index(name='count')
payment_pct     = payment_channel.copy()
totals          = payment_pct.groupby('store_channel')['count'].transform('sum')
payment_pct['pct'] = (payment_pct['count'] / totals * 100).round(1)

print("Payment Method Split by Channel (%):")
print(payment_pct.to_string(index=False))
```

```
Payment Method Split by Channel (%):
store_channel payment_method  count  pct
       Online    Credit Card   2033 44.7
       Online     Debit Card    949 20.9
       Online     Mobile Pay    857 18.8
       Online         PayPal    707 15.5
       Online        Unknown      1  0.0
     Physical           Cash   1104 20.3
     Physical    Credit Card   2148 39.5
     Physical     Debit Card   1389 25.5
     Physical     Mobile Pay    797 14.6
     Physical        Unknown      4  0.1
```

Finding

Physical stores outperform the online channel across all key metrics.

Over the 18-month period:

- **Physical revenue:** ~$360,000
- **Online revenue:** ~$310,000

This creates a **~$50,000 revenue lead** for physical stores.

Net profit shows the same trend:

- **Physical:** ~$220,000
- **Online:** ~$185,000

The monthly trend chart confirms this pattern. **Physical revenue stays higher than online in every month**, although both channels show similar fluctuations.

Average order value is nearly identical for both channels, meaning the higher revenue and profit in physical stores are driven by **more transactions, not higher spending per order.**

Is Marketing Spend Generating Returns?

Analysis type: Diagnostic + Prescriptive (Why? + What to do?)

StyleSense spends money on marketing every month. This section measures whether that spending is actually generating revenue and, more importantly, whether it is generating profit.

Return on Marketing Spend (ROMS) answers the question: for every dollar we spend on marketing, how many dollars of revenue do we get back? A ROMS of 10 means $10 of revenue per dollar spent.

Profit ROMS asks the same question but uses net profit instead of revenue. If revenue ROMS looks healthy but profit ROMS is low, marketing is driving sales at margins too thin to be worthwhile.

The second chart compares sales that happened during a promotion period against sales that happened outside one, to test whether promotions deliver better or worse results than regular selling.

In [18]:
```
# Isolate monthly marketing spend from the expenses dataset
marketing_expenses = expenses[expenses['category'] == 'Marketing'].copy()
monthly_marketing  = marketing_expenses.groupby('month')['amount'].sum().reset_index()
monthly_marketing.columns = ['month', 'marketing_spend']

# Monthly revenue from sales
monthly_revenue = sales.groupby('month')['total_sale'].sum().reset_index()
monthly_revenue.columns = ['month', 'revenue']

# Monthly net profit from sales
monthly_profit = sales.groupby('month')['net_profit'].sum().reset_index()
monthly_profit.columns = ['month', 'net_profit']

# Combine all three into one table
mkt = monthly_marketing.merge(monthly_revenue, on='month', how='inner')
mkt = mkt.merge(monthly_profit, on='month', how='inner')

# ROMS: how much revenue is earned per dollar of marketing spend
mkt['roms'] = (mkt['revenue'] / mkt['marketing_spend']).round(2)

# Profit ROMS: how much net profit is earned per dollar of marketing spend
mkt['profit_roms'] = (mkt['net_profit'] / mkt['marketing_spend']).round(2)

mkt['month_dt'] = mkt['month'].dt.to_timestamp()
```

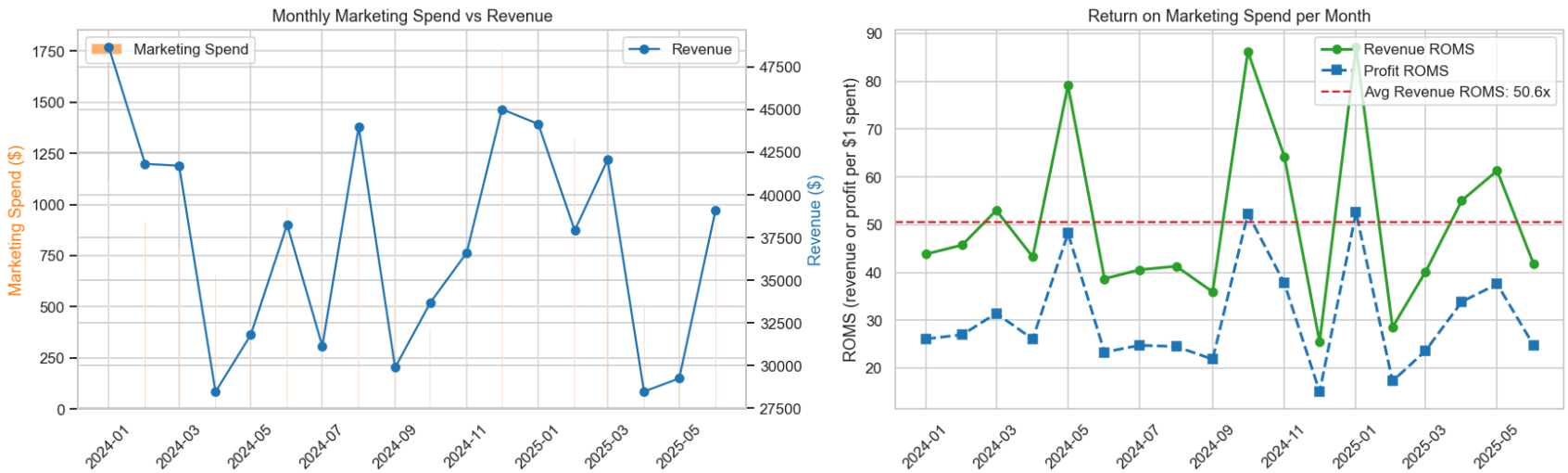
```python
print("Monthly Marketing Spend vs Revenue:")
print(mkt[['month', 'marketing_spend', 'revenue', 'net_profit', 'roms', 'profit_roms']].to_string(index=False))
```

```
Monthly Marketing Spend vs Revenue:
  month  marketing_spend  revenue  net_profit  roms  profit_roms
2024-01          1110.42 48676.10    28985.17 43.84        26.10
2024-02           914.09 41826.37    24669.08 45.76        26.99
2024-03           786.96 41717.20    24755.96 53.01        31.46
2024-04           658.12 28469.56    17164.76 43.26        26.08
2024-05           401.81 31800.97    19374.95 79.14        48.22
2024-06           988.81 38262.66    23073.06 38.70        23.33
2024-07           767.38 31139.84    19007.75 40.58        24.77
2024-08          1064.86 43995.76    26106.18 41.32        24.52
2024-09           831.30 29888.35    18165.56 35.95        21.85
2024-10           390.80 33694.93    20440.62 86.22        52.30
2024-11           569.80 36618.58    21518.62 64.27        37.77
2024-12          1766.78 45025.71    26594.43 25.48        15.05
2025-01           506.40 44155.95    26619.35 87.20        52.57
2025-02          1328.67 37925.25    22951.92 28.54        17.27
2025-03          1048.74 42057.40    24757.75 40.10        23.61
2025-04           517.18 28483.59    17525.45 55.07        33.89
2025-05           477.12 29253.64    17936.06 61.31        37.59
2025-06           935.25 39117.08    23193.08 41.83        24.80
```

In [19]:

```python
# Left chart: marketing spend as bars with revenue as a line (dual axis).
# Right chart: both ROMS metrics over time so we can see if they diverge.
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Left chart: bars for spend, line for revenue on a second y-axis
ax2 = axes[0].twinx()
axes[0].bar(mkt['month_dt'], mkt['marketing_spend'], color=ORANGE, alpha=0.6, label='Marketing Spend')
ax2.plot(mkt['month_dt'], mkt['revenue'], color=BLUE, marker='o', label='Revenue')
axes[0].set_title("Monthly Marketing Spend vs Revenue")
axes[0].set_ylabel("Marketing Spend ($)", color=ORANGE)
ax2.set_ylabel("Revenue ($)", color=BLUE)
axes[0].tick_params(axis='x', rotation=45)
axes[0].legend(loc='upper left')
ax2.legend(loc='upper right')

# Right chart: revenue ROMS and profit ROMS with an average reference line
axes[1].plot(mkt['month_dt'], mkt['roms'],        color=GREEN,  marker='o', linewidth=2, label='Revenue ROMS')
axes[1].plot(mkt['month_dt'], mkt['profit_roms'], color=BLUE,   marker='s', linewidth=2, linestyle='--', label='Prof
```

```
axes[1].axhline(mkt['roms'].mean(), color=RED, linestyle='--',
                label=f"Avg Revenue ROMS: {mkt['roms'].mean():.1f}x")
axes[1].set_title("Return on Marketing Spend per Month")
axes[1].set_ylabel("ROMS (revenue or profit per $1 spent)")
axes[1].legend()
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```



In [20]:

```
# Compare promotional transactions vs non-promotional transactions.
# is_promotion = True means the sale happened during a promotional period.
# We want to know whether promotions lead to better results or just cannibalise margin.
promo = sales.groupby('is_promotion').agg(
    transactions    = ('sale_id',        'count'),
    avg_order_value = ('total_sale',     'mean'),
    avg_net_profit  = ('net_profit',     'mean'),
    avg_units_sold  = ('quantity_sold',  'mean'),
    return_rate     = ('is_returned',    'mean')
).reset_index()

promo['return_rate']     = (promo['return_rate'] * 100).round(1)
promo['avg_order_value'] = promo['avg_order_value'].round(2)
promo['avg_net_profit']  = promo['avg_net_profit'].round(2)
promo['label']           = promo['is_promotion'].map({True: 'Promotion', False: 'No Promotion'})
```

```
print("Promotional vs Non-Promotional Transactions:")
print(promo[['label', 'transactions', 'avg_order_value', 'avg_net_profit',
             'avg_units_sold', 'return_rate']].to_string(index=False))
```

```
Promotional vs Non-Promotional Transactions:
       label  transactions  avg_order_value  avg_net_profit  avg_units_sold  return_rate
No Promotion          7306            68.11           41.32        1.865864          4.9
   Promotion          2683            65.03           37.63        1.879240          4.7
```

Findings:

Marketing Efficiency (ROMS)

Marketing spend is **small relative to revenue** and varies widely by month.
Spending drops close to **zero dollar in some months** (e.g., April 2024 and June 2025) and peaks at about **1,750**.

Across the 18-month period, **average Revenue ROMS is 50.6x**, meaning **$1 in marketing generates about 50.60dollars in revenue**.

- **Best month:** September 2024 (~87x ROMS)
- **Weakest period:** Early 2025 (Jan–Mar), around **35x–40x**

Profit ROMS remains **20–25 points lower than Revenue ROMS**, usually **20x–30x**.
This gap reflects operating costs that reduce revenue before it becomes profit.

September 2024 stands out as the **most efficient month for marketing performance**.

When Should We Restock Seasonal Products?

Analysis type: Predictive (What is likely to happen next?)

StyleSense stocks winter and summer products that have predictable seasonal demand. The problem is that these products have been going out of stock during peak season, meaning sales are being lost because the shelves are empty at exactly the moment customers want to buy.

This section uses 18 months of daily stock data to map out, month by month, when stock levels historically drop the fastest. It also calculates how many days of stock remain at the current rate of sales, which gives a forward-looking warning of when a product will

run out if no order is placed.

A product with fewer than 21 days of stock remaining needs a reorder placed immediately. Most suppliers require 3 weeks (21 days) from order to delivery, so anything below that threshold means stock will run out before the next order arrives.

In [21]:
```
# Merge inventory data with the products table to get the season tag for each product.
# The inventory table records daily stock levels but does not store the season.
inv_products = inventory.merge(products[['product_id', 'season']], on='product_id', how='left')

# Filter to seasonal products only
winter_inv = inv_products[inv_products['season'].isin(['Winter', 'All'])]
summer_inv = inv_products[inv_products['season'].isin(['Summer', 'All'])]

print("Winter products:", inv_products[inv_products['season'] == 'Winter']['product_name'].unique())
print("Summer products:", inv_products[inv_products['season'] == 'Summer']['product_name'].unique())
```

```
Winter products: <StringArray>
[         'Men's Hoodie',   'Men's Puffer Jacket',        'Women's Hoodie',
 'Women's Puffer Jacket',      'Women's Cardigan',           'Kids Jacket']
Length: 6, dtype: str
Summer products: <StringArray>
[    'Men's Polo Shirt', 'Women's Floral Dress',        'Women's Skirt',
           'Kids Dress',    'Men's Swim Shorts',     'Women's Swimsuit']
Length: 6, dtype: str
```

In [24]:
```
# Calculate average stock level for each calendar month.
# We derive month_num directly from the date column to ensure it is always present,
# even if it was not carried through the merge in the previous cell.
inv_products['month_num'] = inv_products['date'].dt.month

# Re-filter the seasonal subsets now that month_num is on inv_products
winter_inv = inv_products[inv_products['season'].isin(['Winter', 'All'])]
summer_inv = inv_products[inv_products['season'].isin(['Summer', 'All'])]

# Group by month number and calculate the mean stock level for each month
winter_monthly = winter_inv[winter_inv['season'] == 'Winter'].groupby('month_num')['stock_level'].mean().reset_index(
summer_monthly = summer_inv[summer_inv['season'] == 'Summer'].groupby('month_num')['stock_level'].mean().reset_index(

# Convert month number to a readable three-letter name (1 becomes Jan, etc.)
winter_monthly['month_name'] = pd.to_datetime(winter_monthly['month_num'], format='%m').dt.strftime('%b')
summer_monthly['month_name'] = pd.to_datetime(summer_monthly['month_num'], format='%m').dt.strftime('%b')
```

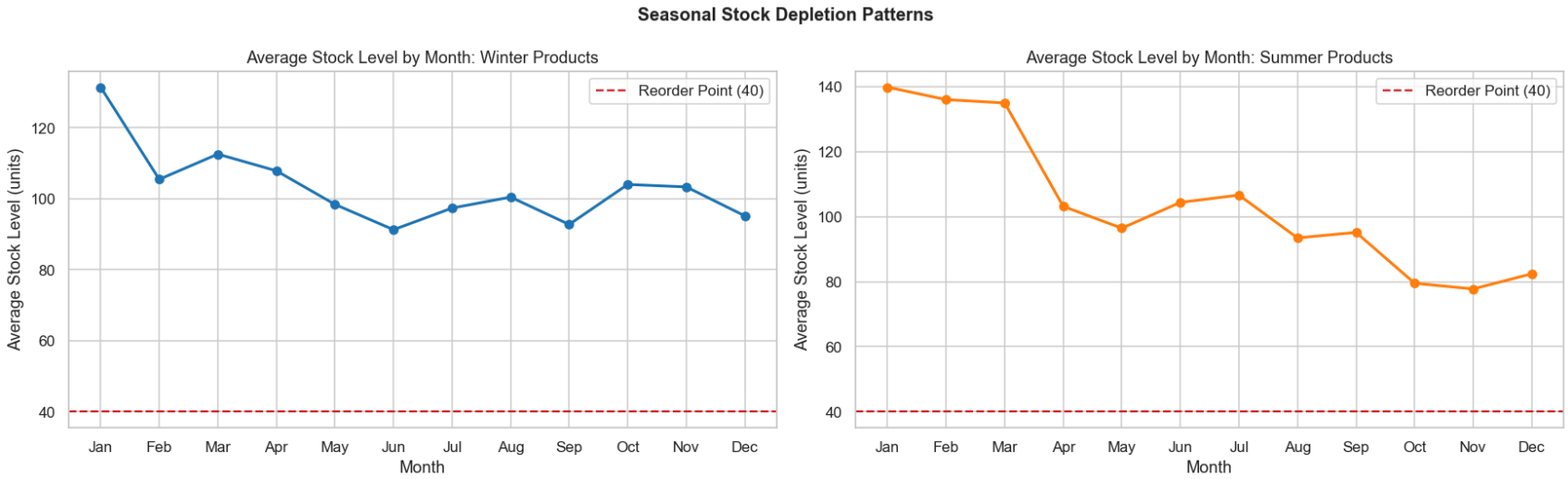
```
print('Average stock level by month for Winter products:')
print(winter_monthly[['month_name', 'stock_level']].to_string(index=False))
print()
print('Average stock level by month for Summer products:')
print(summer_monthly[['month_name', 'stock_level']].to_string(index=False))
```

```
Average stock level by month for Winter products:
month_name  stock_level
       Jan   131.462366
       Feb   105.500000
       Mar   112.602151
       Apr   107.908333
       May    98.438172
       Jun    91.300000
       Jul    97.403226
       Aug   100.489247
       Sep    92.777778
       Oct   104.059140
       Nov   103.361111
       Dec    95.193548

Average stock level by month for Summer products:
month_name  stock_level
       Jan   139.793011
       Feb   135.979532
       Mar   134.943548
       Apr   103.147222
       May    96.435484
       Jun   104.383333
       Jul   106.612903
       Aug    93.451613
       Sep    95.150000
       Oct    79.516129
       Nov    77.761111
       Dec    82.419355
```

In [25]:

```
# Two line charts showing how average stock level changes across the year
# for winter products (left) and summer products (right).
# The red dashed line marks the reorder point at 40 units.
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
```

```
axes[0].plot(winter_monthly['month_name'], winter_monthly['stock_level'],
             marker='o', color=BLUE, linewidth=2)
axes[0].set_title("Average Stock Level by Month: Winter Products")
axes[0].set_ylabel("Average Stock Level (units)")
axes[0].set_xlabel("Month")
axes[0].axhline(40, color=RED, linestyle='--', label='Reorder Point (40)')
axes[0].legend()

axes[1].plot(summer_monthly['month_name'], summer_monthly['stock_level'],
             marker='o', color=ORANGE, linewidth=2)
axes[1].set_title("Average Stock Level by Month: Summer Products")
axes[1].set_ylabel("Average Stock Level (units)")
axes[1].set_xlabel("Month")
axes[1].axhline(40, color=RED, linestyle='--', label='Reorder Point (40)')
axes[1].legend()

plt.suptitle("Seasonal Stock Depletion Patterns", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



In [26]:

```
# For each season and month, count how many days products spent at or below
# the reorder point. Also calculate the average days_until_stockout for those periods.
# days_until_stockout was calculated in feature engineering using a 7-day rolling sales average.

# Filter to seasonal products that are at or below the reorder point
near_stockout_seasonal = inv_products[
```

```python
    inv_products['is_near_stockout'] &
    inv_products['season'].isin(['Winter', 'Summer'])
].copy()

# For days_until_stockout, values of 999 mean zero sales pace so we exclude them
risky = near_stockout_seasonal[near_stockout_seasonal['days_until_stockout'] < 999]

# Count near-stockout days and average days until stockout per season and month
stockout_by_season_month = near_stockout_seasonal.groupby(['season', 'month_num']).agg(
    near_stockout_days = ('date', 'count')
).reset_index()

days_avg = risky.groupby(['season', 'month_num']).agg(
    avg_days_to_stockout = ('days_until_stockout', 'mean')
).reset_index()
days_avg['avg_days_to_stockout'] = days_avg['avg_days_to_stockout'].round(1)

# Combine the two summaries
stockout_by_season_month = stockout_by_season_month.merge(days_avg, on=['season', 'month_num'], how='left')
stockout_by_season_month['month_name'] = pd.to_datetime(
    stockout_by_season_month['month_num'], format='%m').dt.strftime('%b')

print("Near-stockout events by season and month:")
print(stockout_by_season_month[['season', 'month_name', 'near_stockout_days',
                                'avg_days_to_stockout']].to_string(index=False))
```

```
Near-stockout events by season and month:
season month_name  near_stockout_days  avg_days_to_stockout
Summer        Feb                   1                  35.0
Summer        May                   8                  26.0
Summer        Jul                   1                  21.5
Summer        Sep                   1                  16.5
Summer        Oct                   2                  35.0
Summer        Nov                  15                  37.0
Winter        Jan                   4                  16.4
Winter        Feb                   3                  31.8
Winter        Mar                   2                  18.0
Winter        Apr                   1                  93.3
Winter        Sep                   5                 154.0
Winter        Oct                   3                  57.7
Winter        Dec                   1                  12.2
```

Finding: Both seasonal product groups remain well above the 40-unit reorder threshold throughout the year, so there is no immediate stockout crisis visible in this data. However, the depletion trends identify the months where stock is at its most stretched and where the business needs to ensure restocking has already been completed.

For winter products, stock is highest in January at around 130 units and reaches its lowest average in June at approximately 92 units, before stabilising at around 95 to 104 units for the rest of the year. The downward drift from January through to June is the window when winter stock is being drawn down most actively.

For summer products, stock peaks in January at around 140 units and declines sharply from August through to November, hitting its lowest point in November at approximately 79 units before recovering slightly in December. The August to November period is when summer products are most at risk of falling toward the reorder threshold, and it is the period that needs the most attention in the buying calendar.

The reorder point line on both charts sits at 40 units. Neither product group averages below it, but the trends show the direction of travel. Restock orders for summer products should be placed by July so that stock arrives before the August to November drawdown begins. Winter products should be reordered by October ahead of any late-season demand.

Which Products Are Frequently Out of Stock?

Analysis type: Diagnostic + Prescriptive (Why? + What to do?)

Q6 showed when seasonal stock runs low across the whole season. This question zooms in to identify individual products that repeatedly spend the most days at or below the reorder point of 40 units, regardless of whether they are seasonal.

Some products are chronically understocked because the reorder quantities are too small, the reorder trigger is set too late, or supplier lead times are longer than assumed. The heatmap below acts as a reorder calendar: darker red cells identify the exact product and month combinations where StyleSense is most likely to have empty shelves.

In [27]:
```
# Filter inventory to records where stock is at or below the reorder point.
# is_near_stockout is a boolean flag added during feature engineering.
near_stockout = inventory[inventory['is_near_stockout']].copy()

# For each product, count the total days spent near stockout,
# the lowest stock level ever reached, and the average stock level during risky periods.
stockout_summary = near_stockout.groupby(['product_id', 'product_name', 'category']).agg(
```

```
    near_stockout_days = ('date',              'count'),
    min_stock_reached  = ('stock_level',       'min'),
    avg_stock_at_risk  = ('stock_level',       'mean')
).reset_index()

# Separately calculate average days_until_stockout, excluding the 999 placeholder
# (999 was used in feature engineering to represent no near-term stockout risk)
risky_inv = near_stockout[near_stockout['days_until_stockout'] < 999]
days_avg_by_product = risky_inv.groupby('product_name').agg(
    avg_days_to_stockout = ('days_until_stockout', 'mean')
).reset_index()
days_avg_by_product['avg_days_to_stockout'] = days_avg_by_product['avg_days_to_stockout'].round(1)

# Combine with the main summary
stockout_summary = stockout_summary.merge(days_avg_by_product, on='product_name', how='left')
stockout_summary['avg_stock_at_risk'] = stockout_summary['avg_stock_at_risk'].round(1)
stockout_summary = stockout_summary.sort_values('near_stockout_days', ascending=False)

print("Products Most Frequently Near Stockout:")
print(stockout_summary.to_string(index=False))
```

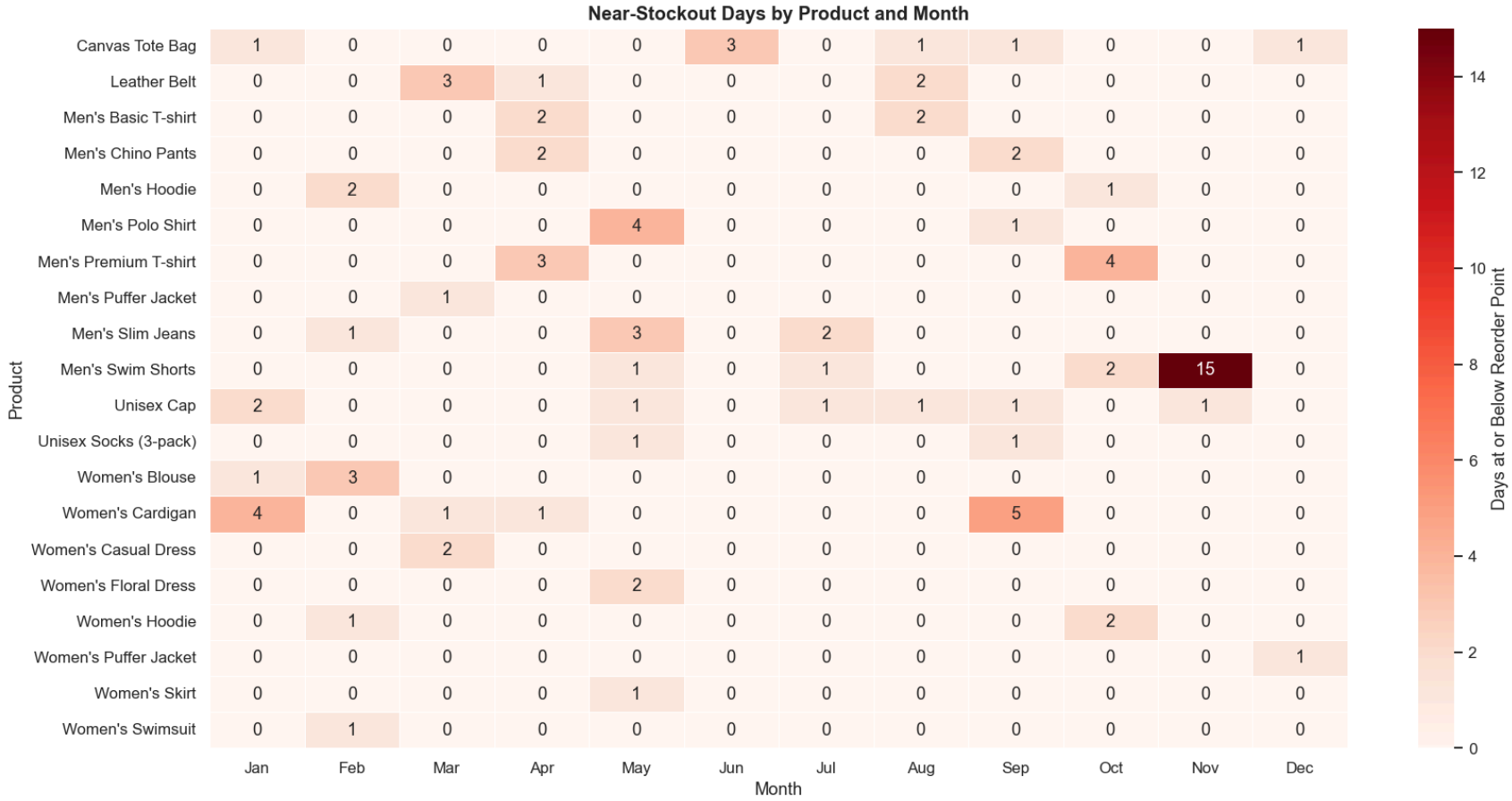
Products Most Frequently Near Stockout:

product_id	product_name	category	near_stockout_days	min_stock_reached	avg_stock_at_risk	avg_days_to_stockout
P024	Men's Swim Shorts	Men	19	40	40.0	36.9
P015	Women's Cardigan	Women	11	40	40.0	85.6
P002	Men's Premium T-shirt	Men	7	40	40.0	92.4
P020	Unisex Cap	All	7	40	40.0	13.4
P023	Canvas Tote Bag	All	7	40	40.0	20.5
P022	Leather Belt	All	6	40	40.0	20.2
P003	Men's Slim Jeans	Men	6	40	40.0	36.8
P007	Men's Polo Shirt	Men	5	40	40.0	20.4
P004	Men's Chino Pants	Men	4	40	40.0	40.0
P001	Men's Basic T-shirt	Men	4	40	40.0	60.6
P010	Women's Blouse	Women	4	40	40.0	24.0
P013	Women's Hoodie	Women	3	40	40.0	34.3
P005	Men's Hoodie	Men	3	40	40.0	55.2
P021	Unisex Socks (3-pack)	All	2	40	40.0	20.1
P008	Women's Floral Dress	Women	2	40	40.0	23.4
P009	Women's Casual Dress	Women	2	40	40.0	20.0
P011	Women's Skirt	Women	1	40	40.0	20.0
P006	Men's Puffer Jacket	Men	1	40	40.0	23.3
P014	Women's Puffer Jacket	Women	1	40	40.0	12.2

```
     P025      Women's Swimsuit    Women                    1                  40                40.0
35.0
```

In [28]:

```python
# Heatmap showing how many days each product spent near or below the reorder point,
# broken down by calendar month. Darker red cells show higher stockout risk.
# This chart acts as a reorder calendar: orders should arrive before the red months.
near_stockout['month_num'] = near_stockout['date'].dt.month

heatmap_data  = near_stockout.groupby(['product_name', 'month_num']).size().reset_index(name='days_at_risk')
heatmap_pivot = heatmap_data.pivot(index='product_name', columns='month_num', values='days_at_risk').fillna(0)

# Replace numeric month columns with readable names
month_labels = {1:'Jan', 2:'Feb', 3:'Mar', 4:'Apr', 5:'May', 6:'Jun',
                7:'Jul', 8:'Aug', 9:'Sep', 10:'Oct', 11:'Nov', 12:'Dec'}
heatmap_pivot.columns = [month_labels.get(c, c) for c in heatmap_pivot.columns]

plt.figure(figsize=(16, 8))
sns.heatmap(heatmap_pivot, annot=True, fmt='.0f', cmap='Reds', linewidths=0.5,
            cbar_kws={'label': 'Days at or Below Reorder Point'})
plt.title("Near-Stockout Days by Product and Month", fontsize=13, fontweight='bold')
plt.xlabel("Month")
plt.ylabel("Product")
plt.tight_layout()
plt.show()
```



In [29]:

```
# Stockout risk summarised at the category level.
# days_per_product shows the average stockout exposure per product in each category.
category_stockout = near_stockout.groupby('category').agg(
    near_stockout_days = ('date',          'count'),
    products_affected  = ('product_name', 'nunique')
).reset_index()

category_stockout['days_per_product'] = (category_stockout['near_stockout_days'] / category_stockout['products_affect
category_stockout = category_stockout.sort_values('near_stockout_days', ascending=False)

print("Stockout Risk by Category:")
print(category_stockout.to_string(index=False))
```

```
Stockout Risk by Category:
category  near_stockout_days  products_affected  days_per_product
     Men                  49                  8               6.1
   Women                  25                  8               3.1
     All                  22                  4               5.5
```

Finding: The overall stockout picture across the product range is mild. The vast majority of cells in the heatmap are white or very light pink, meaning most products spend zero or one or two days at or below the reorder point in any given month.

The one clear exception is Men's Swim Shorts in November, which recorded 15 days at or below the reorder threshold. This is the darkest cell on the entire heatmap and represents a genuine stockout risk during what should be an off-peak month for swimwear. It is worth investigating whether this reflects a delayed clearance pattern or an unexpected demand spike, as November is not the typical peak for swim products.

The next most at-risk situations are Women's Cardigan in January (4 days) and September (5 days), Men's Premium T-shirt in April (3 days) and October (4 days), and Men's Polo Shirt in May (4 days). These represent isolated periods of tight stock rather than a systemic problem.

The category summary table above shows which product categories carry the most cumulative stockout days. For practical planning purposes, Men's Swim Shorts in November needs an earlier reorder deadline, and Women's Cardigan stock levels in January and September should be reviewed before those months arrive.

What Is Our True Profit After All Costs?

Analysis type: Descriptive + Prescriptive (What happened? + What to do?)

The sales data shows gross and net profit at the transaction level, but it does not include the full cost of running the business. Rent, payroll, insurance, software, and marketing are paid every month regardless of how many items are sold. Subtracting those operating costs from sales profit gives the true operating profit: what the business actually earned after every bill was paid.

Operating costs are split into two types. Fixed costs (Rent, Insurance, Software) stay the same every month no matter how much or how little is sold. Variable costs (Cost of Goods Sold, Marketing, Payroll, Utilities) change in proportion to business activity and can be adjusted more easily if revenue falls.

In [30]:
```python
# Total 18-month profit picture.
# net_profit from sales already accounts for COGS and returns at the transaction level.
# We then subtract all operating expenses to get the true operating profit.
total_net_profit_sales = sales['net_profit'].sum()
total_expenses         = expenses['amount'].sum()
total_fixed            = expenses[expenses['is_fixed_cost'] == True]['amount'].sum()
total_variable         = expenses[expenses['is_fixed_cost'] == False]['amount'].sum()
true_operating_profit  = total_net_profit_sales - total_expenses

print(f"Total Net Profit from Sales (after returns) : ${total_net_profit_sales:>12,.2f}")
print(f"  Fixed Operating Expenses                  : ${total_fixed:>12,.2f}")
print(f"  Variable Operating Expenses               : ${total_variable:>12,.2f}")
print(f"Total Operating Expenses                    : ${total_expenses:>12,.2f}")
print(f"True Operating Profit                       : ${true_operating_profit:>12,.2f}")
print(f"Operating Profit Margin                     : {true_operating_profit / sales['total_sale'].sum() * 100:.1f}%"
```

```
Total Net Profit from Sales (after returns) : $  402,839.75
  Fixed Operating Expenses                  : $   72,900.00
  Variable Operating Expenses               : $  474,579.42
Total Operating Expenses                    : $  547,479.42
True Operating Profit                       : $ -144,639.67
Operating Profit Margin                     : -21.5%
```

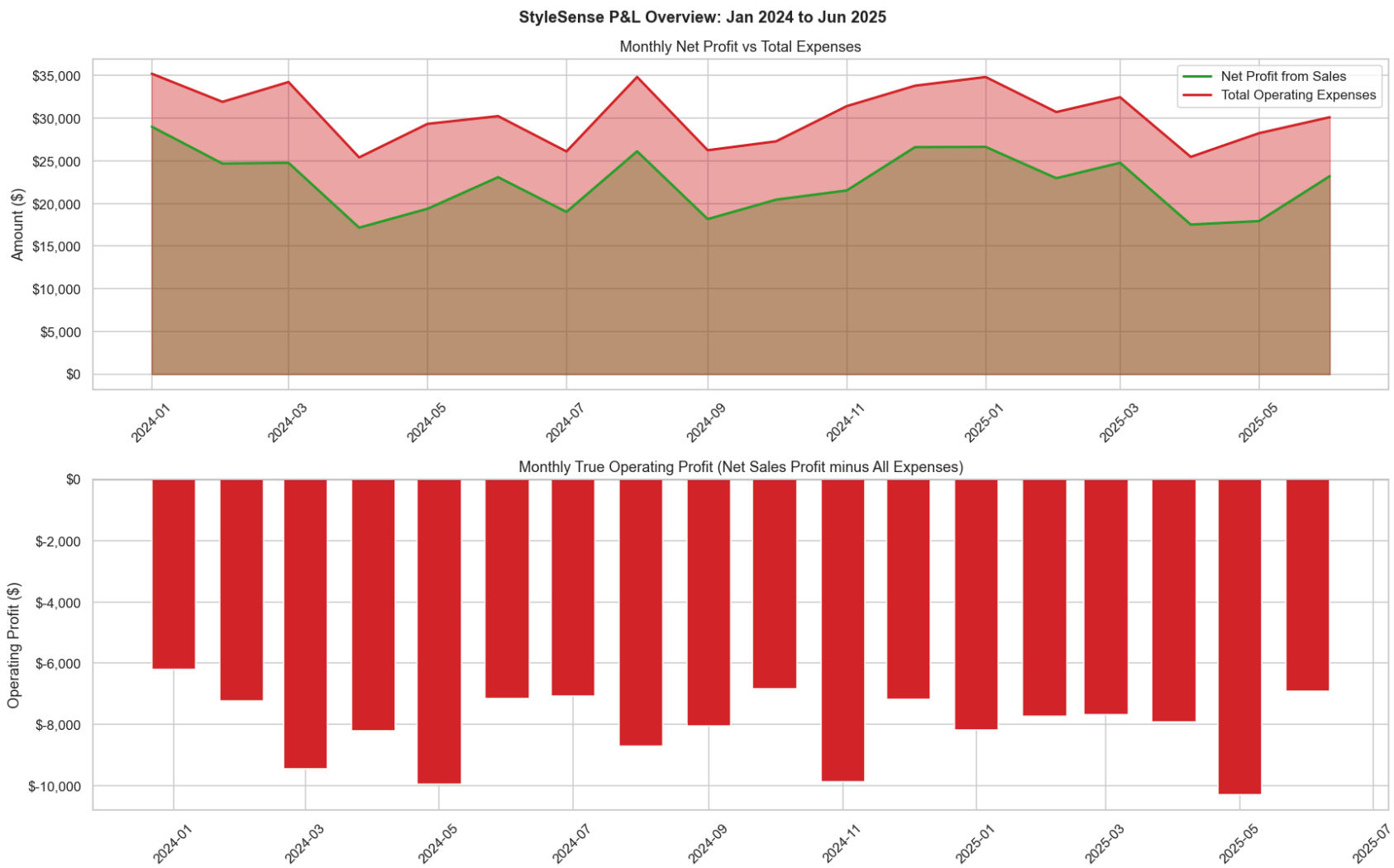
In [31]:
```python
# Monthly true profit: net profit from sales minus all operating expenses.
# This shows which months the business is actually profitable after all costs are paid.
monthly_sales_profit = sales.groupby('month')['net_profit'].sum().reset_index()
monthly_sales_profit.columns = ['month', 'net_profit']

monthly_expenses = expenses.groupby('month')['amount'].sum().reset_index()
monthly_expenses.columns = ['month', 'total_expenses']

# Combine into a monthly P&L table
monthly_pnl = monthly_sales_profit.merge(monthly_expenses, on='month', how='inner')
monthly_pnl['true_profit'] = monthly_pnl['net_profit'] - monthly_pnl['total_expenses']
monthly_pnl['month_dt']    = monthly_pnl['month'].dt.to_timestamp()

print("Monthly P&L Summary:")
print(monthly_pnl[['month', 'net_profit', 'total_expenses', 'true_profit']].to_string(index=False))
```

```
Monthly P&L Summary:
  month  net_profit  total_expenses  true_profit
2024-01    28985.17        35186.46     -6201.29
2024-02    24669.08        31892.38     -7223.30
2024-03    24755.96        34215.46     -9459.50
2024-04    17164.76        25386.92     -8222.16
2024-05    19374.95        29315.40     -9940.45
2024-06    23073.06        30221.78     -7148.72
2024-07    19007.75        26085.15     -7077.40
2024-08    26106.18        34810.01     -8703.83
2024-09    18165.56        26225.41     -8059.85
2024-10    20440.62        27267.85     -6827.23
2024-11    21518.62        31388.59     -9869.97
2024-12    26594.43        33784.47     -7190.04
2025-01    26619.35        34800.35     -8181.00
2025-02    22951.92        30697.41     -7745.49
2025-03    24757.75        32431.75     -7674.00
2025-04    17525.45        25449.45     -7924.00
2025-05    17936.06        28225.54    -10289.48
2025-06    23193.08        30095.04     -6901.96
```

In [32]:

```
# Two-panel chart showing the full profit picture.
# Top panel: net profit from sales vs total expenses over 18 months.
# Bottom panel: true operating profit per month, green for positive and red for negative.
fig, axes = plt.subplots(2, 1, figsize=(16, 10))

# Top panel: filled area chart of profit vs expenses
axes[0].fill_between(monthly_pnl['month_dt'], monthly_pnl['net_profit'],    alpha=0.4, color=GREEN)
axes[0].fill_between(monthly_pnl['month_dt'], monthly_pnl['total_expenses'], alpha=0.4, color=RED)
axes[0].plot(monthly_pnl['month_dt'], monthly_pnl['net_profit'],    color=GREEN, linewidth=2, label='Net Profit from
axes[0].plot(monthly_pnl['month_dt'], monthly_pnl['total_expenses'], color=RED,   linewidth=2, label='Total Operating
axes[0].set_title("Monthly Net Profit vs Total Expenses")
axes[0].set_ylabel("Amount ($)")
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
axes[0].legend()
axes[0].tick_params(axis='x', rotation=45)

# Bottom panel: true operating profit bar chart, coloured green if positive and red if negative
bar_colors = [GREEN if v >= 0 else RED for v in monthly_pnl['true_profit']]
axes[1].bar(monthly_pnl['month_dt'], monthly_pnl['true_profit'], color=bar_colors, width=20)
axes[1].axhline(0, color='black', linewidth=0.8)
axes[1].set_title("Monthly True Operating Profit (Net Sales Profit minus All Expenses)")
```

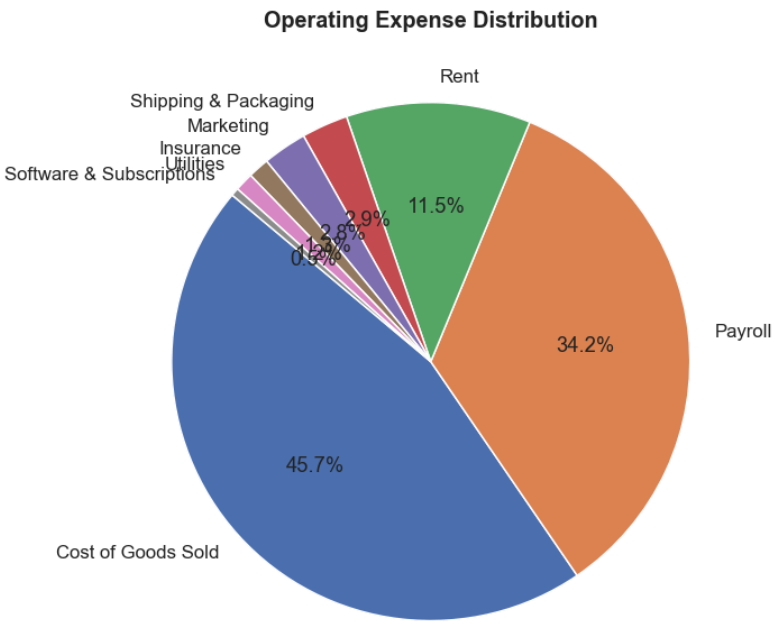
```
axes[1].set_ylabel("Operating Profit ($)")
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
axes[1].tick_params(axis='x', rotation=45)

plt.suptitle("StyleSense P&L Overview: Jan 2024 to Jun 2025", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

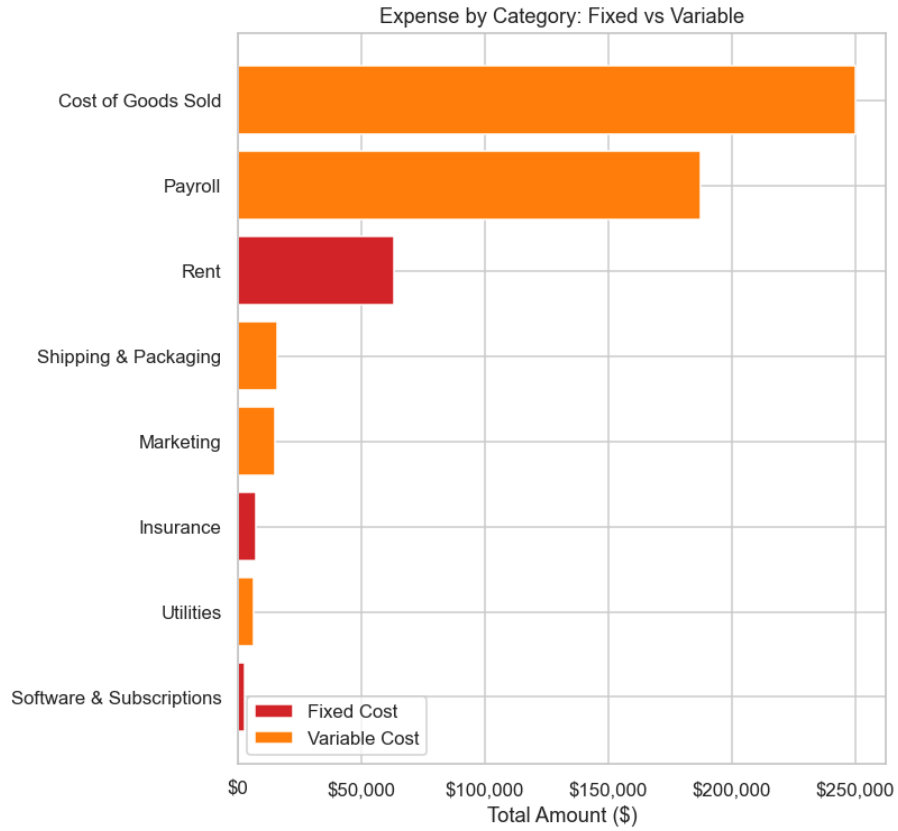


In [33]:
```
# Expense breakdown showing how much was spent in each category
# and whether that category is a fixed or variable cost.
```

```python
expense_breakdown = expenses.groupby(['category', 'is_fixed_cost'])['amount'].sum().reset_index()
expense_breakdown = expense_breakdown.sort_values('amount', ascending=False)
expense_breakdown['pct']       = (expense_breakdown['amount'] / expense_breakdown['amount'].sum() * 100).round(1)
expense_breakdown['cost_type'] = expense_breakdown['is_fixed_cost'].map({True: 'Fixed', False: 'Variable'})

print("Expense Breakdown:")
print(expense_breakdown[['category', 'cost_type', 'amount', 'pct']].to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 7))

# Left chart: pie chart showing the share of each expense category
axes[0].pie(expense_breakdown['amount'], labels=expense_breakdown['category'],
            autopct='%1.1f%%', startangle=140)
axes[0].set_title("Operating Expense Distribution", fontsize=13, fontweight='bold')

# Right chart: horizontal bar chart coloured red for fixed and orange for variable
bar_colors = [RED if f else ORANGE for f in expense_breakdown['is_fixed_cost']]
axes[1].barh(expense_breakdown['category'][::-1], expense_breakdown['amount'][::-1],
             color=bar_colors[::-1])
axes[1].set_title("Expense by Category: Fixed vs Variable")
axes[1].set_xlabel("Total Amount ($)")
axes[1].xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
legend_elements = [mpatches.Patch(facecolor=RED,    label='Fixed Cost'),
                   mpatches.Patch(facecolor=ORANGE, label='Variable Cost')]
axes[1].legend(handles=legend_elements)

plt.tight_layout()
plt.show()
```

```
Expense Breakdown:
                 category cost_type    amount  pct
      Cost of Goods Sold  Variable 249989.00 45.7
                  Payroll  Variable 187300.32 34.2
                     Rent     Fixed  63000.00 11.5
    Shipping & Packaging  Variable  15870.57  2.9
                Marketing  Variable  15064.49  2.8
                Insurance     Fixed   7200.00  1.3
                Utilities  Variable   6355.04  1.2
Software & Subscriptions     Fixed   2700.00  0.5
```





Finding: StyleSense operated at a loss in every single month across the full 18-month period from January 2024 to June 2025. There are no green bars in the bottom panel. Every bar is red. Monthly operating losses range from approximately $2,000 in

the better months to around $10,000 in the worst months, with the deepest losses occurring in March 2024, May 2024, and around May to June 2025.

The top panel shows why: operating expenses (red line) consistently run above net profit from sales (green line) in every month without exception. Expenses range from approximately 26,000 to 35,000 per month while net profit from sales ranges from approximately 17,000 to 29,000. The gap between the two lines is the monthly loss.

The expense breakdown reveals where the money goes. Cost of Goods Sold is the largest expense at 45.7% of all costs, totalling approximately $250,000 over 18 months.

Payroll is the second largest at 34.2%, approximately $190,000.

Together these two categories account for nearly 80% of all operating costs. Rent adds another 11.5% (approximately $60,000). All remaining costs including Shipping and Packaging, Marketing, Insurance, Utilities, and Software together make up less than 5% of total expenses and are not the source of the problem.

The business is not loss-making because of avoidable overhead. It is loss-making because COGS and Payroll together exceed the net profit generated by sales every month. Addressing this requires either growing sales revenue, reducing the cost of goods through better supplier terms, or reviewing payroll relative to revenue capacity.

What StyleSense Should Do

1. Protect the Puffer Jacket range above all else Men's and Women's Puffer Jackets are the profit engine of the business, generating over $62,000 in combined net profit. Ensure they are always in stock, prominently displayed in both channels, and never subject to deep discounts. A stockout or unnecessary markdown on either product has a disproportionate impact on total profitability.

2. Cap discounts at 10% and remove the 20% tier The 20% discount tier generates only around 350 transactions, the fewest of any tier, at the lowest profit margin (54%) and with no reduction in return rates. It delivers no benefit over lower tiers and costs approximately 8 margin points compared to the 0% tier. The data shows customers purchase most at full price. Cap the maximum discount at 10% and redirect promotional investment toward loyalty rewards for the top segments instead.

3. Launch a win-back campaign for At Risk customers using their favourite categories The At Risk list in Q2 ranks customers by historical spend. These customers averaged $2,693 and have gone quiet. Contact them with a personalised message featuring their favourite product category (identified in the Q2 favourite category table). The cost of reactivating a known customer is far lower than acquiring a new one.

4. Invest more in the physical channel

Physical stores generated $50,000 more in revenue and 35,000dollars more in net profit than online over 18 months, and outperformed online in every single month. Any increase in physical store investment, whether in staffing, merchandising, or local marketing, is operating in the higher-performing channel. Online should be maintained but should not receive disproportionate investment at the expense of what is clearly the stronger channel.

5. Concentrate marketing spend in September and surrounding months September 2024 produced an ROMS of approximately 87x, the highest of any month in the dataset. The months around it (August and October) also trend above the 50.6x average.

Shifting a greater share of the variable marketing budget toward August to October each year maximises the return on every dollar spent. Pull back or pause spend in the January to March period when ROMS historically drops toward 35x to 40x.

6. Review summer product buying to ensure stock arrives by July Summer products average around 106 units in stock in July, then decline steadily to approximately 79 units by November. To prevent stock reaching critical levels during that drawdown, restock orders should be placed and received by July at the latest. Currently the depletion trend suggests ordering may be happening too late in the cycle.

7. Address the Men's Swim Shorts November gap immediately Men's Swim Shorts spent 15 days at or below the reorder threshold in November, the only product with a genuine and severe stockout problem visible in the data. Investigate whether November demand is a clearance spike or late-season demand, then set a dedicated reorder trigger for this product that ensures stock arrives before October ends.

8. Tackle COGS and Payroll to move the business toward profitability The business lost money every month for 18 months. The cause is not small overheads. COGS (45.7%) and Payroll (34.2%) together consume nearly 80% of all operating costs. To move toward profitability, StyleSense needs to pursue at least one of three actions: negotiate lower cost prices with suppliers to reduce COGS as a share of revenue, review whether payroll levels are matched to the revenue each store or channel generates, or increase sales revenue to the point where it exceeds the combined COGS and Payroll burden. Cutting insurance, utilities, or software will have negligible impact. The two big cost lines are where the solution must be found.

In [ ]: